MATH 128 - Programming Assignment 3 - Jie Wen

1. Code for `interpolate1.m` (direct Lagrange formula)

```
function yout = interpolate1(f, xin, xout)
%INTERPOLATE1  Lagrange interpolation (naive formula).
%   yout = INTERPOLATE1(f, xin, xout) evaluates the interpolating
%   polynomial P(x) of degree n that satisfies P(x_j) = f(x_j)
%   for the nodes xin(j), using the Lagrange basis
%       L_j(x) = prod_{k≠j} (x - x_k)/(x_j - x_k),
%   and returns P(xout(i)) in yout(i).

    xin  = xin(:);          % ensure column vectors
    xout = xout(:);
    n    = length(xin) - 1; % degree n

    % Precompute f(x_j)
    fx = f(xin);

    % Precompute denominators for L_j(x): d_j = prod_{k≠j} (x_j - x_k)
    d = ones(n+1,1);
    for j = 1:n+1
        for k = 1:n+1
            if k ~= j
                d(j) = d(j) * (xin(j) - xin(k));
            end
        end
    end

    % Evaluate P(x) at all xout
    m = length(xout);
    yout = zeros(m,1);
    for i = 1:m
        x    = xout(i);
        Pi   = 0;
        for j = 1:n+1
            % numerator of L_j(x): prod_{k≠j} (x - x_k)
            num = 1;
            for k = 1:n+1
                if k ~= j
                    num = num * (x - xin(k));
                end
            end
            Lj = num / d(j);
            Pi = Pi + fx(j) * Lj;
        end
        yout(i) = Pi;
    end
end
```

---

2. Code for `interpolate2.m` (barycentric formula)

```
function yout = interpolate2(f, xin, xout)
%INTERPOLATE2  Barycentric Lagrange interpolation.
%   yout = INTERPOLATE2(f, xin, xout) evaluates the same interpolating
%   polynomial as INTERPOLATE1 but using the barycentric formula
%
```

```
%     P(x) = [sum_j λ_j f(x_j)/(x - x_j)] / [sum_j λ_j/(x - x_j)]
%
%   with λ_j = 1 / prod_{k≠j} (x_j - x_k),
%   and with the convention P(x_j) = f(x_j) if x coincides with a node.

    xin  = xin(:);
    xout = xout(:);
    n    = length(xin) - 1;

    % Precompute f(x_j)
    fx = f(xin);

    % Precompute barycentric weights λ_j
    lambda = ones(n+1,1);
    for j = 1:n+1
        for k = 1:n+1
            if k ~= j
                lambda(j) = lambda(j) * (xin(j) - xin(k));
            end
        end
        lambda(j) = 1 / lambda(j);
    end

    m = length(xout);
    yout = zeros(m,1);

    for i = 1:m
        x = xout(i);

        % Check if x coincides with a node (avoid division by zero)
        idx = find(abs(x - xin) < 1e-14, 1);
        if ~isempty(idx)
            yout(i) = fx(idx);
        else
            % barycentric formula
            denom = 0;
            numer = 0;
            for j = 1:n+1
                term = lambda(j) / (x - xin(j));
                denom = denom + term;
                numer = numer + term * fx(j);
            end
            yout(i) = numer / denom;
        end
    end
end
```

3

(a) How the codes work

- `interpolate1` **(naive Lagrange):**
  - Input: function handle `f`, node vector `xin`, evaluation points `xout`.
  - First computes the node values `f(x_j)` and the denominators $d_j = \prod_{k \neq j}(x_j - x_k)$ for each Lagrange basis polynomial $L_j(x)$.
  - For each evaluation point `x`, it computes the numerators $\prod_{k \neq j}(x - x_k)$, forms $L_j(x) = \text{num}/d_j$, and then sums $P(x) = \sum_j f(x_j) L_j(x)$.
  - Complexity is $O((n+1)^2)$ per `xout` point and is numerically unstable for large n, but sufficient for the assignment.
- `interpolate2` **(barycentric):**

- Uses the precomputed barycentric weights $\lambda_j = 1/\prod_{k\neq j}(x_j - x_k)$.
- For each evaluation point `x`, if `x` coincides with a node `x_j`, it simply returns `f(x_j)`.
- Otherwise it evaluates

$$P(x) = \frac{\sum_j \lambda_j f(x_j)/(x - x_j)}{\sum_j \lambda_j/(x - x_j)}.$$

- This formula has the same mathematical result as Lagrange interpolation but is much more stable and efficient in practice.

(b) Plots and observations

```
% run_interpolation.m
% Driver script for Problems (1) and (2):
% - uses interpolate1.m (naive Lagrange)
% - uses interpolate2.m (barycentric)
% - produces 8 figures as required.

clear; clc; close all;

% Function to interpolate
f = @(x) 1 ./ (1 + 9*x.^2);

% Evaluation grid
xout = linspace(-1, 1, 500)';

% ------------------------------
% First four plots: n = 10, 19
% f(x) vs P(x) for uniform & Chebyshev grids
% ------------------------------
for n = [10 19]
    % Nodes
    xinU = linspace(-1, 1, n+1)';          % uniform
    xinC = cos(linspace(-pi, 0, n+1)');    % Chebyshev

    % True values
    ytrue = f(xout);

    % Use either interpolation code for P(x).
    % Here I use interpolate2 (barycentric) to build P(x).
    yU = interpolate2(f, xinU, xout);
    yC = interpolate2(f, xinC, xout);

    % --- Plot for uniform nodes ---
    figure;
    plot(xout, ytrue, 'k-', 'LineWidth', 1.5); hold on;
    plot(xout, yU, 'r--', 'LineWidth', 1.2);
    plot(xinU, f(xinU), 'ro', 'MarkerSize', 4, 'MarkerFaceColor', 'r'); % show nodes
    xlabel('x'); ylabel('y');
    title(sprintf('f(x) and P_n(x), n = %d, uniform nodes', n));
    legend('f(x)', 'P_n(x)', 'nodes', 'Location', 'Best');
    grid on;

    % --- Plot for Chebyshev nodes ---
    figure;
    plot(xout, ytrue, 'k-', 'LineWidth', 1.5); hold on;
    plot(xout, yC, 'b--', 'LineWidth', 1.2);
    plot(xinC, f(xinC), 'bo', 'MarkerSize', 4, 'MarkerFaceColor', 'b'); % show nodes
    xlabel('x'); ylabel('y');
    title(sprintf('f(x) and P_n(x), n = %d, Chebyshev nodes', n));
    legend('f(x)', 'P_n(x)', 'nodes', 'Location', 'Best');
    grid on;
end
```

```
% ------------------------------
% Last four plots: n = 50, 99
% semilogy of errors for both interpolate1 and interpolate2
% for uniform & Chebyshev nodes
% ------------------------------
for n = [50 99]
    xinU = linspace(-1, 1, n+1)';          % uniform
    xinC = cos(linspace(-pi, 0, n+1)');    % Chebyshev

    ytrue = f(xout);

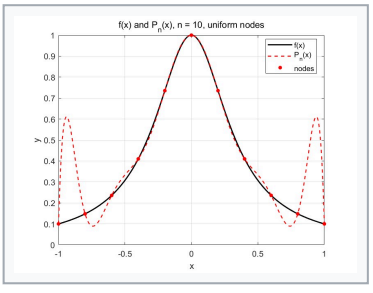
    % ---- Uniform nodes ----
    yU1 = interpolate1(f, xinU, xout);  % naive Lagrange
    yU2 = interpolate2(f, xinU, xout);  % barycentric

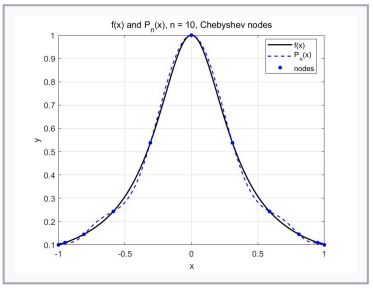
    figure;
    semilogy(xout, 1.0e-18 + abs(yU1 - ytrue), 'r-', 'LineWidth', 1); hold on;
    semilogy(xout, 1.0e-18 + abs(yU2 - ytrue), 'b--', 'LineWidth', 1);
    xlabel('x');
    ylabel('|P_n(x) - f(x)|');
    title(sprintf('Error, n = %d, uniform nodes', n));
    legend('interpolate1 (uniform)', 'interpolate2 (uniform)', ...
           'Location', 'Best');
    grid on;

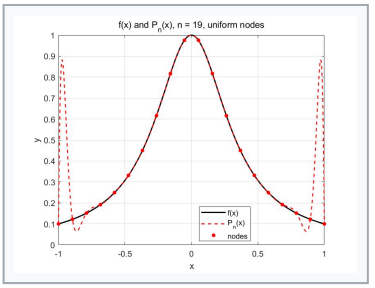
    % ---- Chebyshev nodes ----
    yC1 = interpolate1(f, xinC, xout);
    yC2 = interpolate2(f, xinC, xout);

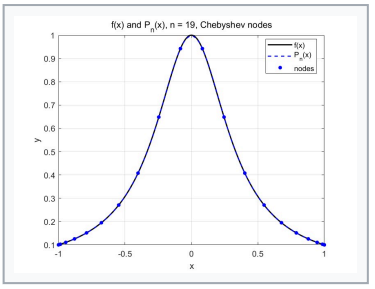
    figure;
    semilogy(xout, 1.0e-18 + abs(yC1 - ytrue), 'r-', 'LineWidth', 1); hold on;
    semilogy(xout, 1.0e-18 + abs(yC2 - ytrue), 'b--', 'LineWidth', 1);
    xlabel('x');
    ylabel('|P_n(x) - f(x)|');
    title(sprintf('Error, n = %d, Chebyshev nodes', n));
    legend('interpolate1 (Chebyshev)', 'interpolate2 (Chebyshev)', ...
           'Location', 'Best');
    grid on;
end

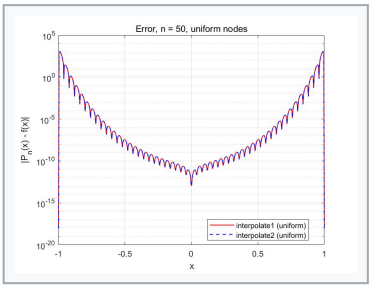
disp('All plots generated.');
```

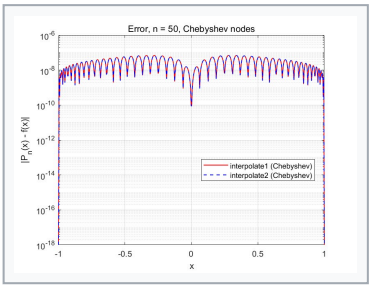


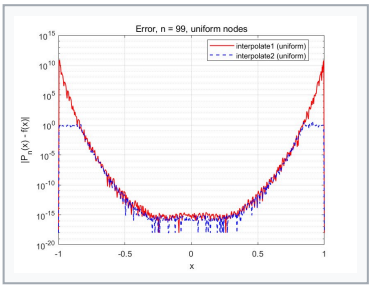


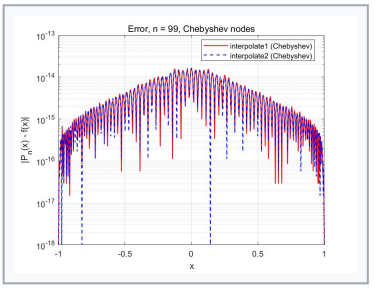

f(x) and $P_n(x)$, n = 19, uniform nodes
y
x
f(x)
$P_n(x)$
nodes


f(x) and $P_n(x)$, n = 19, Chebyshev nodes
y
x
f(x)
$P_n(x)$
nodes


Error, n = 50, uniform nodes
$|P_n(x) - f(x)|$
x
interpolate1 (uniform)
interpolate2 (uniform)


Error, n = 50, Chebyshev nodes
$|P_n(x) - f(x)|$
x
interpolate1 (Chebyshev)
interpolate2 (Chebyshev)


Error, n = 99, uniform nodes
$|P_n(x) - f(x)|$
x
interpolate1 (uniform)
interpolate2 (uniform)


Error, n = 99, Chebyshev nodes
$|P_n(x) - f(x)|$
x
interpolate1 (Chebyshev)
interpolate2 (Chebyshev)